DATABASE TECHNOLOGIES LAB SHEET – 7

Course Code: MCA105P

Topic: Cursors and Triggers

Software: MySQL

PART A – CURSORS

Database Setup

SQL Code:

```
CREATE DATABASE Hospital_Cursor_Trigger_Lab;

USE Hospital_Cursor_Trigger_Lab;

CREATE TABLE Doctors (
   DoctorID INT PRIMARY KEY,
   DoctorName VARCHAR(50),
   Department VARCHAR(40),
   ConsultationFee DECIMAL(10,2),
   Status VARCHAR(20)
);

INSERT INTO Doctors VALUES
(201,'Dr. Anil','Cardiology',1200,'Active'),
(202,'Dr. Beena','Pediatrics',900,'Active'),
(203,'Dr. Chirag','Neurology',1500,'Active'),
(204,'Dr. Divya','Dermatology',1000,'Active'),
(205,'Dr. Eshan','Cardiology',1300,'Active'),
(206,'Dr. Farah','Pediatrics',850,'Active');

CREATE TABLE Doctor_Audit (
   AuditID INT AUTO_INCREMENT PRIMARY KEY,
   DoctorID INT,
   ActionType VARCHAR(30),
   OldFee DECIMAL(10,2),
   NewFee DECIMAL(10,2),
   ActionTime TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE Fee_Log (
   LogID INT AUTO_INCREMENT PRIMARY KEY,
   DoctorID INT,
   DoctorName VARCHAR(50),
   OldFee DECIMAL(10,2),
   NewFee DECIMAL(10,2),
   ChangedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Q1. Display All Doctors Using Cursor

Question: Create a stored procedure using a cursor to display DoctorID and DoctorName for every doctor. Execute it and verify all six doctors are processed.

SQL Code:

```
DELIMITER //

CREATE PROCEDURE DisplayDoctors()
```

```
BEGIN
   DECLARE done INT DEFAULT FALSE;
   DECLARE v_id INT;
   DECLARE v_name VARCHAR(50);

   DECLARE cur CURSOR FOR
   SELECT DoctorID, DoctorName FROM Doctors;

   DECLARE CONTINUE HANDLER FOR NOT FOUND
   SET done = TRUE;

   OPEN cur;

   read_loop: LOOP
      FETCH cur INTO v_id, v_name;

      IF done THEN
         LEAVE read_loop;
      END IF;

      SELECT v_id AS DoctorID, v_name AS DoctorName;
   END LOOP;

   CLOSE cur;
END //

DELIMITER ;

CALL DisplayDoctors();
```

Q2. Doctors with Fee Greater Than 1000

Question: Create a cursor procedure that displays doctors whose ConsultationFee is greater than 1000. Test the procedure.

SQL Code:

```
DELIMITER //

CREATE PROCEDURE HighFeeDoctors()
BEGIN
   DECLARE done INT DEFAULT FALSE;
   DECLARE v_name VARCHAR(50);
   DECLARE v_fee DECIMAL(10,2);

   DECLARE cur CURSOR FOR
   SELECT DoctorName, ConsultationFee
   FROM Doctors
   WHERE ConsultationFee > 1000;

   DECLARE CONTINUE HANDLER FOR NOT FOUND
   SET done = TRUE;

   OPEN cur;

   read_loop: LOOP
      FETCH cur INTO v_name, v_fee;

      IF done THEN
         LEAVE read_loop;
      END IF;

      SELECT v_name AS DoctorName, v_fee AS Fee;
   END LOOP;

   CLOSE cur;
END //
```

```
DELIMITER ;

CALL HighFeeDoctors();
```

Q3. Calculate Total Consultation Fee

Question: Create a cursor procedure with an OUT parameter to calculate the total ConsultationFee of all doctors. Call it and display the OUT value.

SQL Code:

```
DELIMITER //

CREATE PROCEDURE TotalFee(OUT total DECIMAL(10,2))
BEGIN
   DECLARE done INT DEFAULT FALSE;
   DECLARE v_fee DECIMAL(10,2);

   DECLARE cur CURSOR FOR
   SELECT ConsultationFee FROM Doctors;

   DECLARE CONTINUE HANDLER FOR NOT FOUND
   SET done = TRUE;

   SET total = 0;

   OPEN cur;

   read_loop: LOOP
      FETCH cur INTO v_fee;

      IF done THEN
         LEAVE read_loop;
      END IF;

      SET total = total + v_fee;
   END LOOP;

   CLOSE cur;
END //

DELIMITER ;

CALL TotalFee(@total);

SELECT @total AS TotalConsultationFee;
```

Q4. Count Cardiology Doctors

Question: Create a cursor procedure with an OUT parameter to count doctors belonging to the Cardiology department. Execute it and display the result.

SQL Code:

```
DELIMITER //

CREATE PROCEDURE CountCardiology(OUT total INT)
BEGIN
   DECLARE done INT DEFAULT FALSE;
   DECLARE v_id INT;

   DECLARE cur CURSOR FOR
   SELECT DoctorID FROM Doctors
```

```
    WHERE Department = 'Cardiology';

    DECLARE CONTINUE HANDLER FOR NOT FOUND
    SET done = TRUE;

    SET total = 0;

    OPEN cur;

    read_loop: LOOP
        FETCH cur INTO v_id;

        IF done THEN
            LEAVE read_loop;
        END IF;

        SET total = total + 1;
    END LOOP;

    CLOSE cur;
END //

DELIMITER ;

CALL CountCardiology(@count);

SELECT @count AS TotalCardiologyDoctors;
```

Q5. Calculate Average Consultation Fee

Question: Create a cursor procedure to calculate the average ConsultationFee of all doctors. Handle the case where no rows are available.

SQL Code:

```
DELIMITER //

CREATE PROCEDURE AverageFee(OUT avg_fee DECIMAL(10,2))
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE v_fee DECIMAL(10,2);
    DECLARE total DECIMAL(10,2) DEFAULT 0;
    DECLARE cnt INT DEFAULT 0;

    DECLARE cur CURSOR FOR
    SELECT ConsultationFee FROM Doctors;

    DECLARE CONTINUE HANDLER FOR NOT FOUND
    SET done = TRUE;

    OPEN cur;

    read_loop: LOOP
        FETCH cur INTO v_fee;

        IF done THEN
            LEAVE read_loop;
        END IF;

        SET total = total + v_fee;
        SET cnt = cnt + 1;
    END LOOP;

    CLOSE cur;

    IF cnt = 0 THEN
```

```
    SET avg_fee = NULL;
  ELSE
    SET avg_fee = total / cnt;
  END IF;
END //

DELIMITER ;

CALL AverageFee(@average);

SELECT @average AS AverageConsultationFee;
```

Q6. Increase Pediatrics Doctors Fee by 10%

Question: Create a cursor procedure that increases the ConsultationFee of all Pediatrics doctors by 10%. Execute it and verify the Doctors table.

SQL Code:

```
DELIMITER //

CREATE PROCEDURE IncreasePediatricsFee()
BEGIN
  DECLARE done INT DEFAULT FALSE;
  DECLARE v_id INT;

  DECLARE cur CURSOR FOR
  SELECT DoctorID FROM Doctors
  WHERE Department = 'Pediatrics';

  DECLARE CONTINUE HANDLER FOR NOT FOUND
  SET done = TRUE;

  OPEN cur;

  read_loop: LOOP
    FETCH cur INTO v_id;

    IF done THEN
      LEAVE read_loop;
    END IF;

    UPDATE Doctors
    SET ConsultationFee = ConsultationFee * 1.10
    WHERE DoctorID = v_id;
  END LOOP;

  CLOSE cur;
END //

DELIMITER ;

CALL IncreasePediatricsFee();

SELECT * FROM Doctors;
```

Q7. Change Status to Review

Question: Create a cursor procedure that changes Status to 'Review' for doctors whose ConsultationFee is below 1000. Verify the affected rows.

SQL Code:

```
DELIMITER //
```

```
CREATE PROCEDURE UpdateDoctorStatus()
BEGIN
   DECLARE done INT DEFAULT FALSE;
   DECLARE v_id INT;

   DECLARE cur CURSOR FOR
   SELECT DoctorID FROM Doctors
   WHERE ConsultationFee < 1000;

   DECLARE CONTINUE HANDLER FOR NOT FOUND
   SET done = TRUE;

   OPEN cur;

   read_loop: LOOP
      FETCH cur INTO v_id;

      IF done THEN
         LEAVE read_loop;
      END IF;

      UPDATE Doctors
      SET Status = 'Review'
      WHERE DoctorID = v_id;
   END LOOP;

   CLOSE cur;
END //

DELIMITER ;

CALL UpdateDoctorStatus();

SELECT * FROM Doctors;
```

Q8. Cursor-Based Doctor Report

Question: Create a cursor-based doctor report that processes rows ordered by Department and DoctorName and displays DoctorName, Department, and ConsultationFee.

SQL Code:

```
DELIMITER //

CREATE PROCEDURE DoctorReport()
BEGIN
   DECLARE done INT DEFAULT FALSE;
   DECLARE v_name VARCHAR(50);
   DECLARE v_dept VARCHAR(40);
   DECLARE v_fee DECIMAL(10,2);

   DECLARE cur CURSOR FOR
   SELECT DoctorName, Department, ConsultationFee
   FROM Doctors
   ORDER BY Department, DoctorName;

   DECLARE CONTINUE HANDLER FOR NOT FOUND
   SET done = TRUE;

   OPEN cur;

   read_loop: LOOP
      FETCH cur INTO v_name, v_dept, v_fee;

      IF done THEN
```

```
        LEAVE read_loop;
    END IF;

    SELECT v_name AS DoctorName,
           v_dept AS Department,
           v_fee AS ConsultationFee;
  END LOOP;

  CLOSE cur;
END //

DELIMITER ;

CALL DoctorReport();
```

Q9. Count Active Doctors

Question: Create a cursor procedure with an OUT parameter to count all doctors whose Status is 'Active'. Execute it and display the count.

SQL Code:

```
DELIMITER //

CREATE PROCEDURE CountActiveDoctors(OUT total INT)
BEGIN
  DECLARE done INT DEFAULT FALSE;
  DECLARE v_id INT;

  DECLARE cur CURSOR FOR
  SELECT DoctorID FROM Doctors
  WHERE Status = 'Active';

  DECLARE CONTINUE HANDLER FOR NOT FOUND
  SET done = TRUE;

  SET total = 0;

  OPEN cur;

  read_loop: LOOP
    FETCH cur INTO v_id;

    IF done THEN
      LEAVE read_loop;
    END IF;

    SET total = total + 1;
  END LOOP;

  CLOSE cur;
END //

DELIMITER ;

CALL CountActiveDoctors(@total);

SELECT @total AS ActiveDoctors;
```

Q10. Increase Fee by 5%

Question: Create a cursor procedure that gives a 5% fee increase to every doctor whose ConsultationFee is at least 1200. Execute it and verify the updated fees.

SQL Code:

```
DELIMITER //

CREATE PROCEDURE IncreaseDoctorFee()
BEGIN
   DECLARE done INT DEFAULT FALSE;
   DECLARE v_id INT;

   DECLARE cur CURSOR FOR
   SELECT DoctorID FROM Doctors
   WHERE ConsultationFee >= 1200;

   DECLARE CONTINUE HANDLER FOR NOT FOUND
   SET done = TRUE;

   OPEN cur;

   read_loop: LOOP
      FETCH cur INTO v_id;

      IF done THEN
         LEAVE read_loop;
      END IF;

      UPDATE Doctors
      SET ConsultationFee = ConsultationFee * 1.05
      WHERE DoctorID = v_id;
   END LOOP;

   CLOSE cur;
END //

DELIMITER ;

CALL IncreaseDoctorFee();

SELECT * FROM Doctors;
```

PART B – TRIGGERS

Q1. AFTER INSERT Trigger

Question: Create an AFTER INSERT trigger on Doctors that records the new DoctorID, action type, and new ConsultationFee in Doctor_Audit. Insert a test doctor and verify the audit table.

SQL Code:

```
DELIMITER //

CREATE TRIGGER AfterDoctorInsert
AFTER INSERT ON Doctors
FOR EACH ROW
BEGIN
   INSERT INTO Doctor_Audit
   (DoctorID, ActionType, NewFee)
   VALUES
   (NEW.DoctorID, 'INSERT', NEW.ConsultationFee);
END //

DELIMITER ;

INSERT INTO Doctors VALUES
(207, 'Dr. Gaurav', 'Orthopedics', 1100, 'Active');
```

```
SELECT * FROM Doctor_Audit;
```

Q2. AFTER UPDATE Trigger

Question: Create an AFTER UPDATE trigger that records OLD and NEW ConsultationFee values whenever a doctor's fee changes. Update one doctor and verify Doctor_Audit.

SQL Code:

```
DELIMITER //

CREATE TRIGGER AfterFeeUpdate
AFTER UPDATE ON Doctors
FOR EACH ROW
BEGIN
   IF OLD.ConsultationFee <> NEW.ConsultationFee THEN
      INSERT INTO Doctor_Audit
      (DoctorID, ActionType, OldFee, NewFee)
      VALUES
      (NEW.DoctorID, 'UPDATE',
       OLD.ConsultationFee, NEW.ConsultationFee);
   END IF;
END //

DELIMITER ;

UPDATE Doctors
SET ConsultationFee = 1400
WHERE DoctorID = 201;

SELECT * FROM Doctor_Audit;
```

Q3. BEFORE INSERT Trigger – Uppercase Name

Question: Create a BEFORE INSERT trigger that converts DoctorName to uppercase before the doctor is stored. Insert a test doctor using a lowercase name and verify the result.

SQL Code:

```
DELIMITER //

CREATE TRIGGER UppercaseDoctorName
BEFORE INSERT ON Doctors
FOR EACH ROW
BEGIN
   SET NEW.DoctorName = UPPER(NEW.DoctorName);
END //

DELIMITER ;

INSERT INTO Doctors VALUES
(208, 'dr. harsh', 'ENT', 1000, 'Active');

SELECT * FROM Doctors;
```

Q4. Prevent Negative Consultation Fee

Question: Create a BEFORE INSERT trigger that prevents a doctor from being inserted with a negative ConsultationFee. Use SIGNAL SQLSTATE '45000' with a meaningful message and test it.

SQL Code:

```
DELIMITER //
```

```
CREATE TRIGGER CheckDoctorFee
BEFORE INSERT ON Doctors
FOR EACH ROW
BEGIN
   IF NEW.ConsultationFee < 0 THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Consultation fee cannot be negative';
   END IF;
END //

DELIMITER ;

INSERT INTO Doctors VALUES
(209, 'Dr. Test', 'General', -500, 'Active');
```

Q5. AFTER DELETE Trigger

Question: Create an AFTER DELETE trigger that records the deleted DoctorID and OLD ConsultationFee in Doctor_Audit. Delete a test doctor and verify the audit record.

SQL Code:

```
DELIMITER //

CREATE TRIGGER AfterDoctorDelete
AFTER DELETE ON Doctors
FOR EACH ROW
BEGIN
   INSERT INTO Doctor_Audit
   (DoctorID, ActionType, OldFee)
   VALUES
   (OLD.DoctorID, 'DELETE', OLD.ConsultationFee);
END //

DELIMITER ;

DELETE FROM Doctors
WHERE DoctorID = 208;

SELECT * FROM Doctor_Audit;
```

Q6. AFTER UPDATE Trigger – Fee Log

Question: Create an AFTER UPDATE trigger that stores every changed consultation fee in Fee_Log, including DoctorID, DoctorName, OLD fee, and NEW fee. Test it with an UPDATE.

SQL Code:

```
DELIMITER //

CREATE TRIGGER LogFeeChanges
AFTER UPDATE ON Doctors
FOR EACH ROW
BEGIN
   IF OLD.ConsultationFee <> NEW.ConsultationFee THEN
      INSERT INTO Fee_Log
      (DoctorID, DoctorName, OldFee, NewFee)
      VALUES
      (NEW.DoctorID, NEW.DoctorName,
       OLD.ConsultationFee, NEW.ConsultationFee);
   END IF;
END //
```

```
DELIMITER ;

UPDATE Doctors
SET ConsultationFee = 1600
WHERE DoctorID = 203;

SELECT * FROM Fee_Log;
```

Q7. BEFORE UPDATE Trigger – Prevent Negative Fee

Question: Create a BEFORE UPDATE trigger that prevents ConsultationFee from becoming negative. Test the trigger with an invalid UPDATE.

SQL Code:

```
DELIMITER //

CREATE TRIGGER PreventNegativeUpdate
BEFORE UPDATE ON Doctors
FOR EACH ROW
BEGIN
   IF NEW.ConsultationFee < 0 THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Negative fee is not allowed';
   END IF;
END //

DELIMITER ;

UPDATE Doctors
SET ConsultationFee = -100
WHERE DoctorID = 201;
```

Q8. BEFORE UPDATE Trigger – Normalize Status

Question: Create a BEFORE UPDATE trigger that normalizes a Status value of 'active' to 'Active'. Test it by updating a doctor.

SQL Code:

```
DELIMITER //

CREATE TRIGGER NormalizeDoctorStatus
BEFORE UPDATE ON Doctors
FOR EACH ROW
BEGIN
   IF LOWER(NEW.Status) = 'active' THEN
      SET NEW.Status = 'Active';
   END IF;
END //

DELIMITER ;

UPDATE Doctors
SET Status = 'active'
WHERE DoctorID = 201;

SELECT * FROM Doctors;
```

Q9. AFTER UPDATE Trigger – Department Change

Question: Create an AFTER UPDATE trigger that records an audit event when a doctor's Department changes. Change one department and verify the audit table.

SQL Code:

```
CREATE TABLE Department_Audit (
   AuditID INT AUTO_INCREMENT PRIMARY KEY,
   DoctorID INT,
   OldDepartment VARCHAR(40),
   NewDepartment VARCHAR(40),
   ActionTime TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

DELIMITER //

CREATE TRIGGER LogDepartmentChange
AFTER UPDATE ON Doctors
FOR EACH ROW
BEGIN
   IF OLD.Department <> NEW.Department THEN
      INSERT INTO Department_Audit
      (DoctorID, OldDepartment, NewDepartment)
      VALUES
      (NEW.DoctorID, OLD.Department, NEW.Department);
   END IF;
END //

DELIMITER ;

UPDATE Doctors
SET Department = 'Oncology'
WHERE DoctorID = 201;

SELECT * FROM Department_Audit;
```

Q10. Custom Trigger – Doctor Name Validation

Question: Create and test a trigger of your choice that demonstrates a useful hospital data-validation or auditing rule.

SQL Code:

```
DELIMITER //

CREATE TRIGGER ValidateDoctorName
BEFORE INSERT ON Doctors
FOR EACH ROW
BEGIN
   IF NEW.DoctorName IS NULL OR
      TRIM(NEW.DoctorName) = '' THEN

      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Doctor name cannot be empty';

   END IF;
END //

DELIMITER ;

INSERT INTO Doctors VALUES
(210, '', 'General', 1000, 'Active');
```